

Lucas Zacarias Martins Neves

**Deteccção de vulnerabilidades de memória em C:
Abordagem integrando VSA e HOFG**

São Paulo

2026

Lucas Zacarias Martins Neves

Detecção de vulnerabilidades de memória em C: Abordagem integrando VSA e HOFG

Proposta de Trabalho de Conclusão de Curso
apresentado ao SENAC Santo Amaro como
requisito parcial para aprovação na disciplina
de TCC1 do curso de Ciência da computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Bacharelado em Ciência da computação

Orientador: Prof. Nome do Orientador

São Paulo
2026

Resumo

Palavras-chave: Vazamento de memória, HOFG, VSA.

Sumário

1	INTRODUÇÃO	5
1.1	Contexto	5
1.2	Justificativa	5
1.3	Hipótese	6
1.4	Objetivos	7
1.4.1	Objetivo geral	7
1.4.2	Objetivos específicos	7
2	REVISÃO BIBLIOGRÁFICA	9
2.1	Referencial teórico	9
2.1.1	Memória <i>Heap</i>	9
2.1.2	Análise estática	9
2.1.2.1	Árvore sintática abstrata (AST)	10
2.1.2.2	Forma de atribuição única estática (SSA)	10
2.1.2.3	Limitações da análise estática em códigos-fonte	11
2.1.2.4	Relevância da análise estática em códigos-fonte	11
2.1.3	LLVM	12
2.1.3.1	Implementação do Design de três fases	12
2.1.3.2	Representação Intermediária (LLVM IR)	13
2.1.4	Vulnerabilidades de memória	13
2.1.4.1	<i>Common Weakness Enumeration</i> (CWE)	13
2.1.4.2	CWE-401	14
2.1.4.3	CWE-415	15
2.1.4.4	CWE-416	15
2.1.4.5	<i>Common Vulnerabilities and Exposures</i> (CVE)	15
2.2	Metodologia da pesquisa bibliográfica	16
2.3	Fundamentos	16
2.3.1	<i>Heap Object Flow Graph</i> (HOFG)	16
2.3.1.1	Geração do HOFG para o programa inteiro	16
2.3.2	<i>Value-Set Analysis</i>	19
2.3.2.1	<i>Value-Set</i>	19
2.4	Métricas de Avaliação	20
2.4.1	Juliet Test Suite	20
2.4.2	Projetos Open Source	21
2.5	Trabalhos relacionados	21

2.5.1	Trabalho 1: <i>Memory leak detection using Heap Object Flow Graph (HOFG)</i>	21
2.5.2	Trabalho 2: <i>LeakGuard: Detecting Memory Leaks Accurately and Scalably</i>	22
3	DESENVOLVIMENTO	24
3.1	Solução proposta	24
3.2	Cronograma de trabalho	24
	REFERÊNCIAS	25

1 Introdução

1.1 Contexto

Em um levantamento realizado pela *Microsoft Security Response Center* (MSRC) realizado em 2019 (THOMAS, 2019), foi acurado que aproximadamente 70% das vulnerabilidades corrigidas em softwares da multinacional entre o período de 2004 e 2018 tem ligação a falhas de segurança de memória nas linguagens C e C++, causadas por desenvolvedores que inseriram tais erros em seus códigos sem perceber. Além disso, a situação teve piora não somente com o incremento da base de código da empresa, mas também com a adição de mais código aberto aos seus projetos.

É pontuado ainda pela Microsoft que existem métodos para auxiliar os desenvolvedores a desenvolver código seguro, tais métodos variam desde ferramentas e algoritmos desenvolvidos para análise do próprio código escrito até a guias e metodologias de desenvolvimento para software seguro, treinamentos internos, reuniões para monitoria e análise de código. Salvaguardas que atuam em todas as camadas do desenvolvimento de software, e mesmo com tais itens, tais vulnerabilidades de memória continuam predominantes.

Dando destaque às ferramentas de análise de código estático, por vezes exigem tempo para que os desenvolvedores aprendam a fazer uso do analisador, cenário este que torna o uso de tais ferramentas algo inviável quando o tempo não é aliado. Além disso, tais ferramentas não garantem que todas as vulnerabilidades e possíveis falhas de um código sejam encontradas, pois focam numa análise do código-fonte do software, sendo escrito numa linguagem de alto nível e não na linguagem de máquina gerada pelo compilador que será executada pelo hardware (BALAKRISHNAN; REPS, 2010).

Além de métodos de código estático, outra opção também pontuada pela MSRC para a diminuição de ocorrências, seria o uso de linguagens de programação *memory-safe*, ou seja, que retiram das mãos de desenvolvedores os ônus de preocupações com segurança de memória e a adicionam na própria linguagem, fazendo gerenciamento de memória ou empregando regras estritas na fase de compilação, evitando tais vulnerabilidades. Um exemplo de linguagem de programação *memory-safe* é o Rust, sendo desenvolvido pela Mozilla e que tem como um de seus princípios a segurança de memória.

1.2 Justificativa

Apesar da existência de linguagens de programação que integram mecanismos designados para o gerenciamento da memória *Heap*, tais como o C# e Java, e até mesmo com a existência de linguagens de programação que tem como objetivo principal a segurança

e estabilidade do programa, como ocorre com o Rust (THOMAS, 2019), a reescrita de projetos inteiros frequentemente apresenta custos excessivos de portabilidade. Diante disso, a manutenção e o desenvolvimento na linguagem original se tornam necessários, cenário este que pode ocorrer em projetos construídos em linguagens de programação inseguras, como C e C++.

Para garantir maior segurança de tais projetos, um analisador de código pode ser desenvolvido como solução, com foco na detecção de potenciais vazamentos e problemas derivados da má gestão de dados alocados dinamicamente. Embora a análise estática de códigos-fonte apresente limitações — dentre elas, as limitações apresentadas por Balakrishnan (2007) —, tais abordagens continuam sendo úteis no ciclo de desenvolvimento de software, havendo como uma de suas principais justificativas para adoção a possibilidade de identificar e mitigar vulnerabilidades ainda nas fases iniciais do projeto (MØLLER; SCHWARTZBACH, 2018).

Além disso, embora possam ocorrer mudanças no código-fonte originadas em tempo de compilação para arquiteturas específicas, havendo ainda a possibilidade de falhas geradas pelo próprio compilador, a grande maioria das vulnerabilidades relacionadas à segurança de memória são geradas pelos próprios desenvolvedores de forma desavisada (THOMAS, 2019), permanecendo no código-fonte e passíveis de detecção através de algoritmos.

1.3 Hipótese

A hipótese do projeto esta numa integração entre dois métodos de análise estática.

O primeiro método é o *Heap Object Flow Graph* (HOFG) (C; CHERAMANGALATH, 2023), sendo uma representação do fluxo dos blocos de memória alocados na *Heap*, na qual os vértices do grafo são obtidos a partir das variáveis do programa que são ponteiros para a memória *Heap*.

Para automatizar a detecção de vulnerabilidades de memória, foi desenvolvido o *HOFG-Analyser*, uma ferramenta de análise estática que utiliza a representação HOFG para a detecção de tais vulnerabilidades. O processo tem início com a compilação do código-fonte por meio da infraestrutura do *framework* LLVM, com objetivo de gerar a representação intermediária (LLVM IR) da aplicação. A partir dessa versão estruturada na IR, o grafo de fluxo de objetos na memória *Heap* é construído, abrangendo o programa completo. Após a geração completa do grafo, o *HOFG-Analyser* realiza as verificações necessárias a procura de potenciais vazamentos ou falhas de memória (C; CHERAMANGALATH, 2023).

Porém, o *HOFG-Analyser* obteve 14% de falsos positivos em seus testes, apresentando dificuldades com aritmética de ponteiros presente na linguagem C, ocorrendo pois, dado um ponteiro p , as operações $q = p + 1$ e $q = p$ resultam na adição de uma mesma aresta do ponteiro p para o ponteiro q no grafo. Como consequência, as instruções $free(p)$ e $free(q)$ seriam interpretadas de forma simplificada como a liberação do mesmo objeto na memória

Heap (C; [CHERAMANGALATH, 2023](#)).

Para lidar com as dificuldades presentes pela aritmética de ponteiros e ser possível diminuir a taxa de falsos positivos do HOFG original, uma integração com o *Value-Set Analysis* (VSA) ([BALAKRISHNAN, 2007](#)) poderia ser realizada. O VSA, técnica de interpretação abstrata originalmente aplicada para análise de executáveis, tem o objetivo de encontrar uma aproximação segura para o conjunto de valores que cada objeto de dados pode armazenar em cada ponto de um programa.

Ao integrar VSA ao HOFG, seria possível acompanhar o endereço abstrato dos objetos alocados na memória *Heap*, bem como estimar limites de intervalos que tais endereços podem assumir, permitindo assim acompanhar as mudanças de estado dos objetos com mais profundidade, encontrando atribuições que possam, por sua vez, gerar vulnerabilidades de memória.

1.4 Objetivos

Os objetivos deste trabalho dividem-se em um objetivo principal e objetivos secundários, os quais representam passos necessários para que o propósito central seja alcançado.

1.4.1 Objetivo geral

O objetivo principal do trabalho é implementar um analisador estático de código para a linguagem C, com foco na detecção de vulnerabilidades de memória, através da integração da representação *Heap Object Flow Graph* (HOFG) com técnicas de *Value-Set Analysis* (VSA) para o rastreamento preciso de ponteiros (*offset* e *range*).

1.4.2 Objetivos específicos

1. Fundamentar teoricamente a integração do *Value-Set analysis* à estrutura do *Heap Object Flow Graph* (HOFG), demonstrando como essa abordagem conjunta pode mitigar as limitações de rastreamento de ponteiros e reduzir a taxa de falsos positivos da análise estática.
2. Implementar motor de análise estática processando a representação intermediária LLVM IR, construindo os algoritmos necessários para a extração e geração do HOFG e sua integração com o VSA.
3. Avaliar a precisão e a eficiência do analisador implementado utilizando conjuntos de testes que exploram falhas da linguagem C (como vazamentos de memória e usos após desalocação).
4. Comparar os resultados obtidos com referências comparativas tendo como foco principal a taxa de falsos positivos e verdadeiro positivos, e comparar o tempo e

custo computacional da implementação original do HOFG, para validar o impacto da integração.

2 Revisão Bibliográfica

A revisão bibliográfica deste artigo está estruturada nas subseções 2.1 Referencial teórico, contendo os principais conceitos, teorias e definições que constituem a base fundamental deste trabalho; 2.2 Metodologia da pesquisa bibliográfica, abrangendo a forma como a pesquisa bibliográfica foi conduzida; 2.3 Fundamentos, sendo a subseção que contém de forma detalhada conceitos técnicos que utilizados na busca dos objetivos; 2.4 Métricas de avaliação, detalhando como a solução proposta será avaliada e o que será levado em consideração; e, por fim, 2.5 Trabalhos relacionados, subseção que contém breve análise de três artigos com objetivos similares, destacando suas características, resultados obtidos e, principalmente, as distinções em relação a este trabalho.

2.1 Referencial teórico

2.1.1 Memória *Heap*

A memória *Heap* é uma área da memória virtual de um processo utilizada para a alocação dinâmica de memória. Tal região tem início logo após a área de dados não inicializados do programa e cresce para cima, em direção a endereços de memória mais altos. Para cada processo, o *kernel* mantém uma variável *brk* (sendo pronunciada *break*, ou quebra) que aponta para o topo da *Heap* (BRYANT; O'HALLARON, 2010). Um alocador dinâmico gerencia a *Heap* como uma coleção de blocos de tamanhos variados. Cada bloco é um segmento (*chunk*) contínuo de memória virtual que pode estar em dois estados: alocado ou livre. Um bloco alocado é reservado para uso de forma explícita pela aplicação, permanecendo neste estado até que seja liberado, tanto de forma explícita, pelo aplicativo, quanto de forma implícita, pelo próprio alocador de memória (como em sistemas com *Garbage Collection*). E, por sua vez, um bloco livre está disponível para ser alocado, e permanecesse assim até que, de forma explícita, seja alocado através de uma nova requisição de alocação feita pela aplicação. O fato de que, por vezes, o tamanho exato de certas estruturas de dados que serão utilizados em um programa só se tornam conhecidos até que o mesmo seja executado se torna o principal motivo para a utilização da alocação dinâmica de memória e, consequentemente, da memória *Heap* (BRYANT; O'HALLARON, 2010).

2.1.2 Análise estática

A análise estática é uma técnica para obter informações sobre o comportamento em tempo de execução de um programa, porém sem a necessidade de executar o programa.

Isso ocorre pois os métodos de análise estática exploram o comportamento do programa a partir de abstrações e aproximações para mapear o espaço de estados do programa, dessa forma detectando comportamentos tais como vulnerabilidades e fluxos inválidos para qualquer entrada possível.

2.1.2.1 Árvore sintática abstrata (AST)

Segundo [Aho et al. \(2009\)](#), as árvores sintáticas abstratas (ASTs), ou simplesmente árvores sintáticas, possuem para uma expressão, cada nó interno de uma expressão representa um operador, enquanto os seus nós filhos representam os respectivos operandos. Além disso, as ASTs se assemelham às árvores de derivação até certo nível, diferenciando-se pelo fato de que os nós interiores da AST representam construções de programação, ao passo que na árvore de derivação os nós interiores representam símbolos não-terminais. Outra diferença é a ausência de símbolos não-terminais auxiliares, tais como aqueles que representam termos, fatores, ou outras variações de expressões, na AST. Essa falta ocorre pois tais detalhes sintáticos são desnecessários para a fase de tradução, resultando em uma representação mais condensada e focada na semântica do código-fonte.

2.1.2.2 Forma de atribuição única estática (SSA)

O SSA (*Static Single Assignment*) é uma forma de representação de código intermediário, tem como ponto chave as características de que cada definição de variável possui um nome distinto, e de que cada uso refere-se a apenas uma única definição, sendo essas também as duas restrições da representação, significando que, caso um programa atenda tais restrições, tal programa estará em sua forma SSA. Para garantir essa singularidade, variáveis que sofrem múltiplas atribuições no código-fonte são renomeadas, sendo criadas versões distintas para cada nova atribuição (por exemplo, v_1 , v_2 , v_3) ([SINGER, 2022](#)).

Além disso, em programas com desvios do fluxo de controle, contendo blocos condicionais e laços de repetição, diferentes versões da mesma variável podem chegar ao mesmo ponto de convergência no Grafo de Fluxo de Controle (CFG). São inseridas então, nesses pontos de junção, as chamadas funções ϕ (*phi functions*), possuindo N argumentos correspondentes aos N caminhos do fluxo de controle convergentes, e seleciona a versão correta da variável com base no caminho de execução tomado pelo programa. Sendo criada inicialmente para facilitar o desenvolvimento de transformações de programa de alto nível, a forma ganhou popularidade pelas suas propriedades que permitem a simplificação de algoritmos e a redução da complexidade computacional das análises de fluxo de dados. Atualmente, é amplamente utilizada por compiladores de código aberto e comerciais, como é o caso do LLVM, sendo uma estrutura fundamental para a análise estática de programas ([SINGER, 2022](#)).

2.1.2.3 Limitações da análise estática em códigos-fonte

Os métodos de análise estática não garantem que todas as vulnerabilidades de um código-fonte sejam encontradas. Dentre os motivos, sendo listados por [Balakrishnan \(2007\)](#), estão:

- O uso de bibliotecas já compiladas (tais como as DLLs) em programas, por vezes sem estar disponíveis em sua forma fonte. Gerando a necessidade de realizar esboços que simulam os efeitos das chamadas de função de tais bibliotecas. Por não poder realizar a análise em tais bibliotecas, é possível que a mesma possua erros, não sendo detectados durante a análise estática sendo realizada, resultando em inconsistências.
- O fato de que códigos-fonte são modificados pelo compilador durante a compilação, realizando otimizações e adicionando instrumentações de código, sendo itens não encontrados pelas ferramentas que analisam tais códigos.
- O código-fonte de um programa pode ser escrito em mais de uma linguagem de programação, tornando mais dificultosa o design de tais ferramentas de análise, pois precisam dar suporte a múltiplas linguagens de programação, cada uma contendo suas características e peculiaridades.
- E, mesmo em um código-fonte escrito em apenas uma linguagem de programação de alto nível, tal código pode conter código escrito em *assembly* em certos locais. Ferramentas de análise estática normalmente ignoram o código *assembly* adicionado, ou não levam a análise além do código *assembly* adicionado.

2.1.2.4 Relevância da análise estática em códigos-fonte

Mesmo com as limitações presentes nos métodos de análise estática de códigos-fonte — dentre elas, as citadas por [Balakrishnan \(2007\)](#) —, tais abordagens continuam sendo úteis no ciclo de desenvolvimento de software. Um dos principais motivos para sua adoção é o fato de que tais métodos permitem que vulnerabilidades sejam encontradas ainda nas fases iniciais do projeto, viabilizando a correção prévia e evitando custos elevados nas etapas finais ou em produção ([MØLLER; SCHWARTZBACH, 2018](#)).

Além disso, apesar do código-fonte ser modificado em tempo de compilação através das otimizações e instrumentações e existindo a possibilidade de erros gerados pelo compilador para arquiteturas específicas, a maioria das falhas reside no código original. Sendo apontado pela Microsoft, na qual 70% das falhas corrigidas adereçadas a um CVE em seus softwares tinham como origem vulnerabilidades de memória geradas pelos próprios desenvolvedores, estando presentes e detectáveis em seus respectivos códigos-fonte ([THOMAS, 2019](#)).

Por fim, quanto ao uso de bibliotecas pré-compiladas, bem como a mescla de múltiplas linguagens de programação e trechos escritos em linguagem *assembly*, os métodos de

análise estática costumam adotar abordagens conservadoras (isto é, seguras). Ao serem encontrados, tais chamadas para funções e trechos inalcançáveis geram alertas durante a análise, lidando com o desconhecido como se pertencesse ao pior cenário possível. Embora essa característica gere falsos positivos durante a avaliação, é cumprido o propósito de evitar falsos negativos. Garantindo, dessa forma, que nenhum ponto cabível de vulnerabilidade passe despercebido e que erros reais sejam tratados ([MØLLER; SCHWARTZBACH, 2018](#)).

2.1.3 LLVM

Segundo [Lattner e Adiv \(2004\)](#), o LLVM é uma infraestrutura de compilação designada para prover transparência e análise e transformação de programas de forma persistente durante todo o seu ciclo de vida. O sistema é fundamentado em uma representação intermediária (IR) de baixo nível, baseada na sua forma SSA, permitindo uma ponte universal entre diferentes linguagens e arquiteturas. A arquitetura introduz recursos fundamentais para a análise de código:

- Um simples sistema de tipagem independente de linguagem que expõe os tipos primitivos usados comumente para implementar atributos de linguagens de alto nível.
- Instrução para endereçamento aritmético, sendo diferente do C puro, o LLVM utiliza uma instrução específica para cálculos de ponteiros que preserva informações de tipo e limites de estruturas, sendo essencial para detectar acessos inválidos.
- Mecanismo uniforme de exceções, utilizado para implementar recursos de tratamento de exceções comumente encontrados em linguagens de alto nível de forma eficiente.

2.1.3.1 Implementação do Design de três fases

Num compilador moderno, três componentes principais são aplicados ([LATTNER, 2011](#)), sendo eles:

- O front end, responsável por analisar o código-fonte, chegar erros e construir a árvore de sintaxe abstrata (AST) específica para a linguagem em questão.
- O otimizador, responsável por realizar transformações de código numa tentativa de melhorar o tempo de execução do código, realizando, por exemplo, remoções de itens redundantes que seriam computados.
- O back end, também conhecido como o gerador de código. Responsável por mapear o código para o conjunto de instruções da arquitetura alvo. Dentre os itens comumente encontrados no back end, estão inclusos a seleção de instruções, alocação de registradores e o escalonamento de instruções.

Com tal padrão a portabilidade de um compilador para uma nova linguagem de programação fonte gera a necessidade de implementação do novo *front end*, mas permitindo que o otimizador e o *back end* sejam reutilizados. Porém, esse benefício não fora alcançado em todos os casos já que implementações do padrão de três fases, numa época em que o LLVM estava tendo seu início, em linguagens como Perl, Python, Ruby e Java não compartilhavam código entre si, ou seja, cada uma destas linguagens possuía sua própria implementação de *front end*, otimizador e *back end* (LATTNER, 2011).

2.1.3.2 Representação Intermediária (LLVM IR)

A representação intermediária é o aspecto principal do LLVM, sendo a forma que compilador utiliza para representar o código-fonte sendo compilado. É designada para análises de nível intermediário e transformações que são encontradas no setor de otimização de um compilador. Desenhada com objetivos de adicionar suporte para otimizações em tempo de execução não custosas, otimizações interprocedurais, análise completa do programa, transformações abrangendo nível de reestruturação, dentre outros, tem como aspecto mais importante o fato de ser definida como uma linguagem de primeira classe com semântica bem definida, tendo seu conjunto de instruções próximo do padrão RISC e possuindo como um de seus pilares a forma SSA (LATTNER, 2011).

Sendo a única interface para o otimizador do LLVM, se torna o único requisito para escrever um *front end* para a infraestrutura. Por possuir uma forma textual legível, desenvolver um front end que tem como saída a representação intermediária do LLVM se torna algo possível e viável. Tal característica é um dos principais fatores para a disseminação da infraestrutura do LLVM em diferentes aplicações, já que a completude presente na representação permite que o suporte de uma nova linguagem seja simplificado (LATTNER, 2011).

2.1.4 Vulnerabilidades de memória

Segundo o *Microsoft Security Response Center* (MSRC) (THOMAS, 2019), entre o período de 2004 a 2018 aproximadamente 70% das vulnerabilidades corrigidas mapeadas a um CVE foram causadas por desenvolvedores que de forma desavisada adicionaram problemas relacionados a vulnerabilidades de memória em seus códigos escritos nas linguagens C e C++. Além disso, a medida que a base de código da Microsoft era incrementada e mais código aberto era utilizado, tal problema continuava a aumentar.

2.1.4.1 Common Weakness Enumeration (CWE)

A *Common Weakness Enumeration* (CWE) consiste em uma lista desenvolvida de forma colaborativa, que relaciona fraquezas recorrentes em software e hardware. No contexto de segurança, uma fraqueza (*weakness*) é uma condição num software, firmware, hardware,

ou serviço que, a partir de certas circunstâncias, pode contribuir para a introdução de vulnerabilidades. Fraquezas são identificadas, classificadas, descritas e integradas à lista oficial da CWE (*CWE List*). A compreensão dessas fraquezas que resultam em vulnerabilidades permite que desenvolvedores de software, designers de hardware, e arquitetos de segurança atuem na mitigação e eliminação de tais fraquezas ainda nas fases iniciais do ciclo de desenvolvimento ([The MITRE Corporation, 2026a](#)).

A lista CWE é atualizada de três a quatro vezes por ano, incluindo novas descobertas e atualizando itens registrados anteriormente. Antes da publicação oficial, o conteúdo em desenvolvimento é disponibilizado e pode ser acompanhado através do *CWE Content Development Repository* (CDR), sendo um repositório público com objetivo de garantir transparência e colaboração da comunidade, viabilizando a submissão de novas fraquezas, contribuições para registros existentes e acompanhamento das operações internas da CWE ([The MITRE Corporation, 2026a](#)).

Para o escopo deste trabalho, serão consideradas primariamente três vulnerabilidades presentes na lista oficial da CWE, diretamente relacionadas a falhas de memória. São elas:

- **CWE-401:** *Missing Release of Memory after Effective Lifetime*
- **CWE-415:** *Double Free*
- **CWE-416:** *Use After Free*

2.1.4.2 CWE-401

Conforme a [OWASP Foundation \(2026\)](#), um vazamento de memória é uma forma não intencionada de consumo de memória, ocorrendo quando o desenvolvedor falha em liberar um bloco de memória alocada quando a mesma não é mais necessária. Sendo dividido, de forma geral, em três cenários:

- Aplicações de curta duração (*Short Lived User-Land Application*), tendo pouco ou nenhum impacto notável, levando em consideração que sistemas operacionais modernos obtêm a memória perdida após o programa ser encerrado.
- Aplicações de longa duração (*Long Lived User-Land Application*), tendo riscos potenciais já que tais programas continuam perdendo memória ao longo do tempo, causando, eventualmente, o consumo integral dos recursos da memória RAM.
- Processos em nível de núcleo (*Kernel-land Process*), sendo o caso com nível mais alto de consequências, trazendo vazamentos de memória a nível de *Kernel*, gerando problemas de estabilidade voltados ao sistema.

2.1.4.3 CWE-415

Com o título de liberação dupla (*double free*), ocorre quando o mesmo endereço de memória é liberado mais de uma vez. Isso acontece pois, quando o programa chama a função de liberação (como a função *free()* da biblioteca *stdlib*, na linguagem C) utilizando o mesmo endereço de memória como argumento, a estrutura de dados responsável por gerenciar a heap pode ser corrompida, permitindo assim que um usuário malicioso escreva em posições arbitrárias de memória. O impacto de tal corrupção pode causar desde a falha abrupta do programa até a alteração do fluxo de execução. Ao sobrescrever registradores ou endereços específicos de memória, um atacante pode alterar o programa, sendo capaz de injetar e executar códigos arbitrários, resultando, por exemplo, na escalada de privilégios sobre o sistema em um emulador de terminal (OWASP, 2026).

2.1.4.4 CWE-416

O *Use After Free* é a referenciação de um endereço de memória que foi desalocado, tal ato gera consequências imprevisíveis no sistema, com resultados que variam desde uma falha abrupta do programa até a execução de códigos e comandos arbitrários, dependendo do contexto e do momento que a vulnerabilidade ocorre. A reutilização de um endereço desalocado, segundo o *Open Web Application Security Project* (OWASP, 2026), é a forma mais simples de corrupção de dados, visto que, neste cenário, o bloco de memória anteriormente liberado pode ser realocado para um novo objeto válido ao longo da execução do programa. Com isso o ponteiro original ainda aponta para esse endereço (se tornando um ponteiro pendente, ou *dangling pointer*), qualquer alteração feita através dele corromperá os dados da nova alocação legítima, gerando a falha imprevisível.

2.1.4.5 Common Vulnerabilities and Exposures (CVE)

Criado em 1999 pela corporação MITRE, o programa CVE (*Common Vulnerabilities and Exposures*) visa padronizar a identificação e catalogação de vulnerabilidades de segurança cibernética na indústria de tecnologia ([The MITRE Corporation, 2026b](#)).

A Lista CVE (*CVE List*) atua como um dicionário onde cada entrada documenta uma falha real detectada em uma versão específica de um software. Após a confirmação, a vulnerabilidade recebe um identificador único composto pelo ano de relato ou atribuição, seguido por um sequencial numérico. Após o processo de adição de uma entrada ao dicionário, plataformas como o *National Vulnerability Database* (NVD) incrementam o registro com dados críticos, como descrição técnica, índice de severidade metrificado pela CVSS (*Common Vulnerability Scoring System*) e causa raiz da falha proveniente da Lista CWE ([The MITRE Corporation, 2026b](#)).

2.2 Metodologia da pesquisa bibliográfica

Design Science Research

2.3 Fundamentos

2.3.1 *Heap Object Flow Graph* (HOFG)

O *Heap Object Flow Graph* é uma representação de programa baseada nos blocos de memória alocados na *Heap*. Os vértices de tal grafo são constituídos pelas variáveis do programa que atuam como ponteiro para esses blocos de memória. Para converter um programa em sua representação HOFG, é necessário que seja primeiramente convertido para a forma SSA. Tal feito é necessário pois a forma SSA permite a visualização de forma explícita da definição e usos de cada variável, auxiliando assim, na distinção entre as suas atribuições e seus usos (C; CHERAMANGALATH, 2023).

Além disso, as definições e usos indiretos dos objetos também precisam ser levados em consideração para variáveis cujos endereços são referenciados ou para ponteiros de níveis superiores. Tais dados são capturados por meio das informações de ponteiro embutidas no HOFG. A incorporação de informações de ponteiros sem um módulo de análise de ponteiros separado é uma das peculiaridades do HOFG. A análise estática presente no *Heap Object Flow Graph* rastreia o fluxo de objetos alocados na *Heap* para verificar se tais itens alcançam, ou não, um nó de liberação de memória (C; CHERAMANGALATH, 2023).

Além das informações de definições e usos, o HOFG também incorpora o fluxo de controle, o que adiciona sensibilidade de fluxo à análise. A geração do HOFG e sua análise seguem os seguintes passos (C; CHERAMANGALATH, 2023):

- Geração do HOFG para o programa inteiro.
- Poda de caminhos sem vazamentos.
- Detecção de vazamentos a partir do HOFG podado.

2.3.1.1 Geração do HOFG para o programa inteiro

A implementação do HOFG tem como base a infraestrutura do *framework* LLVM. Inicialmente o código-fonte é compilado, arquivo por arquivo, para a obtenção de sua versão *bytecode* (.bc). Em seguida, tais arquivos são vinculados em tempo de ligação (*link time*) com o uso do *LLVM Gold Plugin*, fornecendo, desta maneira, um arquivo *bitcode* unificado representando o programa completo. Dessa forma, o LLVM auxilia na garantia de um dos requisitos da análise de códigos-fonte, sendo a necessidade de lidar com dependências entre múltiplos módulos de código-fonte (C; CHERAMANGALATH, 2023).

Algoritmo 1 Geração do Grafo de Fluxo de Objetos na *Heap* (HOFG)**Entrada:** Função F na forma de Representação Intermediária (IR)**Saída:** Grafo $HOFG(V, E)$ atualizado com os vértices e arestas da função

```

1: Função CONSTRUIRHOFG( $F$ )
2:   for cada bloco básico  $B$  na Função  $F$  do
3:     for cada instrução  $I$  no bloco básico  $B$  do
4:       if  $I$  é uma instrução de alocação de memória da forma:  $p = \text{malloc}(o)$  then
5:         Adicionar os vértices  $o_i$  e  $p$  ao conjunto  $V$  e a aresta  $o_i \rightarrow p$  a  $F$ 
6:       end if
7:       if  $I$  é uma instrução de liberação de memória:  $\text{free}(p)$  then
8:         if  $p \in V$  then
9:           Adicionar aresta de  $p$  para um novo nó free
10:        end if
11:      end if
12:      if  $I$  é uma instrução de cópia da forma:  $q = p$  then
13:        if  $p \in V$  then
14:          if  $q \notin V$  then
15:            Adicionar vértice  $q$  a  $V$ 
16:          end if
17:          Adicionar aresta de fluxo  $p \rightarrow q$  a  $F$ 
18:        end if
19:      end if
20:      if  $I$  é uma instrução de desreferenciamento de ponteiro (ex:  $*p$ ) then
21:        if  $p \in V$  then
22:          if  $*p \notin V$  then
23:            Adicionar  $*p$  a  $V$ 
24:          end if
25:          Adicionar aresta de desreferenciamento  $p \rightarrow *p$  a  $R$ 
26:        end if
27:      end if
28:      if  $I$  é uma instrução de carregamento (load):  $q = *p$  then
29:        if  $*p \in V$  then
30:          if  $q \notin V$  then
31:            Adicionar  $q$  a  $V$ 
32:          end if
33:          Adicionar aresta de fluxo  $*p \rightarrow q$  a  $F$ 
34:        end if
35:      end if
36:      if  $I$  é uma instrução de armazenamento (store):  $*p = q$  then
37:        if  $q \in V$  then
38:          if  $*p \notin V$  then
39:            Adicionar  $*p$  a  $V$ 
40:          end if
41:          Adicionar aresta de fluxo  $q \rightarrow *p$  a  $F$ 
42:        end if
43:        if  $p \rightarrow q \in F$  e as arestas  $p \rightarrow *p, q \rightarrow *q \in R$  then
44:          Adicionar aresta de fluxo derivado  $*p \rightarrow *q$  em  $D$  com as mesmas restrições
45:          de caminho que  $p \rightarrow q$ 
46:        end if
47:      end for
48:    end for
49: end Função

```

Este *bitcode* unificado é então convertido para a forma SSA a partir da chamada do passe *mem2reg pass*, nativo do LLVM. Por fim, o arquivo binário de todo o programa, agora no formato SSA, se torna entrada para o programa responsável por analisar o HOFG e verificar os vazamentos de memória (C; CHERAMANGALATH, 2023).

O HOFG captura o fluxo de cada bloco de memória alocado na *Heap* através das variáveis (ponteiros) presentes no programa. Trata-se de uma representação seletiva, que captura apenas as partes do código que possuem envolvimento com a memória alocada na *Heap* e nos caminhos que essa memória percorre até alcançar diferentes pontos de execução. Cada atribuição de uma variável de ponteiro é registrada na representação, desde que tenha como origem um bloco alocado na *Heap*. A representação de programa foi projetada de tal sorte que toda e qualquer informação de ponteiros necessária para a detecção de erros esta armazenada na sua própria estrutura (C; CHERAMANGALATH, 2023).

A geração do HOFG é iniciada pela geração de sumários de funções. Tais sumários registram as propriedades da função que afetam os componentes do HOFG, sendo uma etapa fundamental para a análise interprocedural (C; CHERAMANGALATH, 2023).

O processo da geração de sumários realiza as seguintes etapas para cada função analisada (C; CHERAMANGALATH, 2023):

- Geração do HOFG local da função.
- Ao encontrar uma instrução de chamada de função, é gerado o sumário da função chamada (*callee*), se o mesmo não estiver disponível.
- Aplicação do sumário da função chamada, se estiver disponível. Esta etapa envolve a adição de arestas direcionadas para (ou a partir de) argumentos reais e formais.
- Atualização do sumário da função atual.

O *Heap Object Flow Graph* é definido como um grafo orientado $HOFG(V, E)$, onde (C; CHERAMANGALATH, 2023):

$$E = \{\text{arestas de fluxo } F, \text{arestas de desreferenciamento } R, \\ \text{e arestas de fluxo derivado } D\} \quad (2.1)$$

$$V = \{\text{objetos alocados na memória Heap } o_i, \text{variáveis de programa } p, p^*, \dots, \\ \text{nós de liberação } free_i\} \quad (2.2)$$

O grafo do HOFG é construído de acordo com as regras descritas no 1. As condições que anotam as arestas do HOFG são derivadas das transições entre os blocos básicos e dos identificadores de blocos básicos que possuem associação com as instruções do programa (C; CHERAMANGALATH, 2023).

O número de vértices presente no HOFG depende da quantidade real de instruções que possuem ponteiros para a memória *Heap* como operandos. Considerando N como o

total de linhas de código (LoC) do programa, o pior caso a ocorre quando cada instrução possui um objeto alocado na memória *Heap* como operando, o que limita a complexidade de espaço a $O(N)$ (C; CHERAMANGALATH, 2023).

Enquanto isso, o número de arestas no grafo depende do caminho de fluxo do objeto, isto é, do número de vezes que o seu valor é utilizado ou copiado após a alocação. Dessa forma, esse número varia para cada objeto alocado no programa. E, assim como ocorre com o número de vértices, a complexidade espacial referente ao número máximo de arestas também pode ser delimitado superiormente por $O(N)$, pois, num pior caso, é considerado como o número total de linhas de código (LoC) no programa (C; CHERAMANGALATH, 2023).

2.3.2 Value-Set Analysis

O *Value-Set Analysis* (VSA) é uma técnica de interpretação abstrata de código executável, que tem como objetivo encontrar uma aproximação segura para o conjunto de valores que cada objeto de dados pode armazenar em cada ponto de um programa (BALAKRISHNAN, 2007). O VSA possui similaridades com o problema de análise de ponteiros (*pointer-analysis*), frequentemente estudado em análises de programas escritos em linguagens de alto nível. Enquanto a análise de ponteiros tradicional determina uma sobreaproximação (*over-approximation*) de conjunto de variáveis cujos endereços uma determinada variável pode conter, o VSA atua determinando uma sobreaproximação do conjunto de endereços que cada objeto de dados pode conter em cada ponto do programa.

Como consequência, os resultados do VSA podem ser utilizados para identificar quais *a-locs* (*abstract locations*) estão contidos nos endereços de um determinado *a-loc*. Por outro lado, o VSA também possui características de análises estáticas numéricas. Além de rastrear endereços, o algoritmo determina uma sobreaproximação do conjunto de valores inteiros que cada objeto de dados pode assumir ao longo da execução (BALAKRISHNAN, 2007).

2.3.2.1 Value-Set

Para representar a junção de endereços de memória e valores numéricos, o VSA utiliza uma estrutura matemática chamada *Value-Set* (conjunto de valores). Um *Value-Set* é representado com uma tupla de intervalos com passo (*strided intervals*, ou SI) de k -bits, simbolizando o conjunto de deslocamentos (*offsets*) em cada região de memória. Um intervalos com passo de k -bit $s[l, u]$ representa o conjunto de números inteiros (BALAKRISHNAN, 2007):

$$\{i \in [-2^{k-1}, 2^{k-1}] \mid l \leq i \leq u, i \equiv l \pmod{s}\} \quad (2.3)$$

A partir disso:

- s é chamado de passo (*stride*).
- $[l, u]$ é chamado de intervalo.
- $0[l, l]$ representa um conjunto unitário.
- $\perp$ também sendo considerado um intervalo com passo, denota um conjunto vazio de deslocamentos.

Considere o conjunto de endereços:

$$S = (Global \rightarrow \{1, 3, 5, 9\}), (AR_{Main} \rightarrow \{-48, -40\}) \quad (2.4)$$

O *Value-Set* de S é o conjunto:

$$(Global \rightarrow 2[1, 9]), (AR_{Main} \rightarrow 8[-48, -40]) \quad (2.5)$$

Em tal cenário, o *Value-Set* de S é uma *over-approximation*, pois o *Value-Set* contempla o endereço global 7, que não é um elemento de S . Além disso, um *Value-Set* é capaz de representar tanto um conjunto de endereços de memória, quanto um conjunto de valores numéricos. Dessa forma, a 2-tupla $(2[1, 9], \perp)$ indica o conjunto de valores numéricos $\{1, 3, 5, 7, 9\}$, assim como indica o conjunto de endereços $(Global, 1), (Global, 3), \dots, (Global, 9)$ (BALAKRISHNAN, 2007).

2.4 Métricas de Avaliação

2.4.1 Juliet Test Suite

O Juliet Test Suite é uma coleção de programas escritos em C/C++ e Java, sendo casos de teste sintéticos, isto é, foram desenvolvidas como exemplos de vulnerabilidades caracterizadas.

A coleção foi desenvolvida pela National Security Agency's Center for Assured Software, estando em domínio público. Segue uma estrutura focada em funções disponíveis na plataforma subjacente e não em bibliotecas de terceiros. Fora desenvolvida no Microsoft Windows, porém boa parte dos programas teste foram desenvolvidos sem a utilização de funções provenientes de uma plataforma ou sistema operacional específica, e, além disso, todos os casos de teste foram compilados.

2.4.2 Projetos Open Source

2.5 Trabalhos relacionados

2.5.1 Trabalho 1: *Memory leak detection using Heap Object Flow Graph* (HOFG)

O artigo que apresenta a representação de programas HOFG, apresenta também o *HOFG-Analyser*, um analisador de código estático baseado nessa estrutura para a detecção de vazamentos de memória. A ferramenta foi implementado sobre o LLVM *framework* versão 12 e o compilador Clang 3.8. Conforme destacado pelos próprios autores, o analisador assume certas concessões em sua corretude (*unsound trade-offs*), de forma similar como ocorre em abordagens anteriores (C; CHERAMANGALATH, 2023).

O primeiro ponto ocorre em caminhos que são finalizados com outra chamada de função de biblioteca que não seja a função *free()*, em tais casos o rastreamento do objeto alocado na *Heap* é interrompido. Além disso, é pontuado que operações aritméticas complexas envolvendo ponteiros não são rastreadas com precisão. Isto é, dado um ponteiro p , as operações $q = p + 1$ e $q = p$ resultam na adição de uma mesma aresta do ponteiro p para o ponteiro q no HOFG. Consequentemente, as instruções *free(p)* e *free(q)* seriam interpretadas de forma simplificada como a liberação do mesmo objeto na memória *Heap* (C; CHERAMANGALATH, 2023).

O *HOFG-Analyser* foi comparado com a ferramenta de código aberto INFER, utilizando oito programas de referência do SPECINT2006 e três projetos de código aberto. A análise classifica as vulnerabilidades encontradas em duas categorias, possíveis vazamentos (*may-leak points*), sendo percursos no HOFG que tem seu destino um nó de liberação, mas que possuam arestas anotadas com condições que só podem ser resolvidas em tempo de execução. E vazamentos confirmados (*must-leak points*), sendo percursos presentes no HOFG que não terminam em um nó de liberação (C; CHERAMANGALATH, 2023).

Diante das limitações relacionadas ao tratamento de aritmética de ponteiros no *HOFG-Analyser* a integração com o *Value-Set Analysis* (VSA) é apresentada como uma abordagem viável. A fragilidade encontrada no HOFG pode ser superada ao se utilizar as aproximações de valores (presentes em intervalos válidos) que cada variável pode assumir em tempo de execução, sendo calculadas através do VSA. Logo, em uma operação como $q = p + 1$, o VSA tem a capacidade de verificar se o deslocamento calculado resulta em um endereço válido que a variável q possa assumir. Ao calcular a diferença de endereços abstratos (*a-locs*) entre p e q , torna-se viável validar ou impedir a criação de uma aresta interligando tais ponteiros no HOFG. Com isso, a integração de HOFG e VSA atuaria como um filtro, capaz de reduzir a imprecisão da análise realizada pelo *HOFG-Analyser*.

2.5.2 Trabalho 2: *LeakGuard: Detecting Memory Leaks Accurately and Scalably*

O *LeakGuard* tem como objetivo a detecção de vazamentos de memória de forma escalável e precisa, possui fluxo de análise dividido em quatro partes (LIANG et al., 2025):

- Fase 1: Pré-processamento do projeto sob teste. Realizado pois softwares reais possuem diversos arquivos de código fonte e configuração de compilação, contendo regras e comandos para a compilação do projeto completo. Enquanto o projeto é compilado, o *LeakGuard* armazena o processo num banco de dados de compilação, sendo utilizado posteriormente.
- Fase 2: Modelagem das funções de alocação e desalocação (*memory allocation/deallocation* - MAD). Detectadas por meio de um processo iterativo, inicializado pela consulta ao grafo de chamadas com objetivo de identificar todas as funções que invocam funções MAD padrão já conhecidas. Após isso, é empregada uma análise de fluxo de dados sensível ao caminho (*path-sensitive*) para mapear novas funções MAD dentro destas funções chamadoras, refinando os respectivos sumários. Esta etapa é realizada até que nenhum sumário adicional seja incorporado.
- Fase 3: Reconhecimento de funções chamadoras. As funções que realizam chamadas para as funções MAD identificadas na fase 2 são filtradas. Todas as funções que não lidam com operações de memória são ignoradas, partindo do pressuposto lógico de que não pode haver vazamentos em funções que não alocam ou liberam memória.
- Fase 4: Detecção de vazamentos. É aplicada a execução simbólica sub-restrita (*under-constrained symbolic execution*) e uma análise sensível ao caminho nas funções chamadoras filtradas na fase anterior. Permitindo que haja foco direto da execução simbólica sobre os pontos de interesse. A ferramenta rastreia o ciclo de vida da memória dinâmica e utiliza um *solver SMT* para verificar se o caminho que gera a falha é realmente viável. Além disso, um mecanismo de análise de escape de ponteiros foi desenvolvido, baseado no fluxo de dados, confirma se a memória foi de fato abandonada ou se seu controle foi transferido para uma variável global, estrutura de dados, parâmetros ou retornos de funções.

Possui como diferencial a modelagem de funções MAD personalizadas, sendo uma das principais motivações para o desenvolvimento da ferramenta. Visto que outras ferramentas apresentam dificuldades na modelagem de tais funções pelo frequente emprego de análises insensíveis ao caminho, que mesclam ramificações do código em uma tentativa de evitar a explosão de estados da função (*path explosion*). Com isso, o próprio rastreamento de ponteiros se torna impreciso, pois as condições reais de atribuições e operações são perdidas ou desconsideradas (LIANG et al., 2025).

Em contraste, o *LeakGuard* realiza uma análise de fluxo de dados de baixo para cima, sensível ao caminho e ao campo (*field-sensitive*), permitindo rastrear com precisão operações de memória aninhadas e identificar se as alocações dependem de parâmetros condicionais em tempo de execução ([LIANG et al., 2025](#)).

3 Desenvolvimento

3.1 Solução proposta

3.2 Cronograma de trabalho

Referências

- AHO, A. V. et al. *Compiladores: Princípios, Técnicas e Ferramentas*. 2. ed. [S.l.]: Pearson Addison-Wesley, 2009. Tradução de Daniel Vieira. 10
- BALAKRISHNAN, G. PhD thesis, *WYSINWYX: What You See Is Not What You Execute*. Madison, WI: [s.n.], 2007. Disponível em: <https://research.cs.wisc.edu/wpis/papers/balakrishnan_thesis.pdf>. 6, 7, 11, 19, 20
- BALAKRISHNAN, G.; REPS, T. Wysinwyx: What you see is not what you execute. *ACM Trans. Program. Lang. Syst.*, Association for Computing Machinery, New York, NY, USA, v. 32, n. 6, ago. 2010. ISSN 0164-0925. Disponível em: <<https://doi.org/10.1145/1749608.1749612>>. 5
- BRYANT, R. E.; O'HALLARON, D. R. *Computer Systems: A Programmer's Perspective*. 2. ed. Upper Saddle River, NJ: Prentice Hall, 2010. 9
- C, H. M.; CHERAMANGALATH, U. Memory leak detection using heap object flow graph (hofg). Association for Computing Machinery, New York, NY, USA, 2023. Disponível em: <<https://doi.org/10.1145/3578527.3578528>>. 6, 7, 16, 18, 19, 21
- LATTNER, C. LLVM. In: BROWN, A.; WILSON, G. (Ed.). *The Architecture of Open Source Applications*. Lulu.com, 2011. v. 1, cap. 11. Acesso em: 13 abr. 2026. Disponível em: <<https://aosabook.org/en/v1/llvm.html>>. 12, 13
- LATTNER, C.; ADVE, V. Llm: A compilation framework for lifelong program analysis & transformation. In: *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization*. USA: IEEE Computer Society, 2004. (CGO '04), p. 75. ISBN 0769521029. 12
- LIANG, H. et al. *LeakGuard: Detecting Memory Leaks Accurately and Scalably*. 2025. Disponível em: <<https://arxiv.org/abs/2504.04422>>. 22, 23
- MØLLER, A.; SCHWARTZBACH, M. I. *Static Program Analysis*. 2018. Department of Computer Science, Aarhus University, <http://cs.au.dk/~amoeller/spa/>. 6, 11, 12
- OWASP Foundation. *Memory leak*. 2026. Acesso em: 13 abr. 2026. Disponível em: <https://owasp.org/www-community/vulnerabilities/Memory_leak>. 14
- SINGER, J. Introduction. In: RASTELLO, F.; Bouchez Tichadou, F. (Ed.). *SSA-based Compiler Design*. [S.l.]: Springer International Publishing, 2022. ISBN 978-3-030-80515-9. 10
- The MITRE Corporation. *About CWE*. 2026. Acesso em: 13 abr. 2026. Disponível em: <<https://cwe.mitre.org/about/index.html>>. 14
- The MITRE Corporation. *About the CVE Program*. 2026. Acesso em: 03 mai. 2026. Disponível em: <<https://www.cve.org/About/Overview>>. 15
- THOMAS, G. *A proactive approach to more secure code*. 2019. Acesso em: 13 abr. 2026. Disponível em: <<https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code>>. 5, 6, 11, 13